



Cloud Security with AWS IAM



Muhammad Aqib

Policy editor Visual | JSON

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "ec2:*",
7       "Resource": "*",
8       "Condition": {
9         "StringEquals": {
10          "ec2:ResourceTag/Env": "development"
11        }
12      }
13    },
14    {
15      "Effect": "Allow",
16      "Action": "ec2:Describe*",
17      "Resource": "*"
18    },
19    {
20      "Effect": "Deny",
21      "Action": [
22        "ec2:DeleteTags",
23        "ec2:CreateTags"
24      ],
25      "Resource": "*"
26    }
27  ]
28 }
```

Edit statement

Select an existing statement to edit or add a new statement

+



Introducing Today's Project!

Project overview

In this project, I will demonstrate how to use AWS IAM to control access and permission settings in our AWS account. I'm doing this project to learn about cloud security from the absolute foundations. Every company thinks about access permissions, and there are even entire jobs called 'IAM Engineers' focused on the skills we are about to learn today.

Tools and concepts

Services I used were Amazon EC2 and AWS IAM. Key concepts I learnt include IAM users, policies, user groups, and account aliases. We also learned how to use the policy simulator and how JSON policies work. We also learned how to launch instances, how to tag an instance, and how to log in as another user.

Project reflection

This project took me approximately 2 hours today. The most challenging part was understanding the IAM policies. What was most rewarding to me was seeing the permission denied when our intern tried to delete the production instance.



Tags

What I did in this step

In this step, I will launch two EC2 instances because we need to boost Nextwork's computing power. We are expecting more users and traffic on our website over the summer break!

Understanding tags

Tags are organizational tools that let us label our resources.

My tag configuration

The tag I've used on my EC2 instances is called env, which stands for environment. The values I've assigned for my instances are production and development.



Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

▼ Name and tags [Info](#)

Key [Info](#)

Q Name



Value [Info](#)

Q nextwork-dev-aqib



Resource types [Info](#)

Select resource types



Remove

Instances



Key [Info](#)

Q env



Value [Info](#)

Q development



Resource types [Info](#)

Select resource types



Remove

Instances



Add new tag

You can add up to 48 more tags.



IAM Policies

What I did in this step

In this step, I will use new IAM policies to control the access level of a nextwork intern because they should have access to the development environment but not the production environment.

Understanding IAM policies

IAM Policies are the rules that define who can do what in our AWS Account.

The policy I set up

For this project, I've set up a policy using JSON.

Policy effect

I've created a policy that allows the policy holder to have permission to do anything they want to any instance tagged with 'development'.



Muhammad Aqib

NextWork Student

nextwork.ai

Understanding Effect, Action, and Resource

The Effect, Action, and Resource attributes of a JSON policy means whether or not the policy is allowing or denying action.

My JSON Policy

Policy editor

Visual | JSON

1 {
2 "Version": "2012-10-17",
3 "Statement": [
4 {
5 "Effect": "Allow",
6 "Action": "ec2:*",
7 "Resource": "*",
8 "Condition": {
9 "StringEquals": {
10 "ec2:ResourceTag/Env": "development"
11 }
12 }
13 },
14 {
15 "Effect": "Allow",
16 "Action": "ec2:Describe*",
17 "Resource": "*"
18 },
19 {
20 "Effect": "Deny",
21 "Action": [
22 "ec2:DeleteTags",
23 "ec2:CreateTags"
24],
25 "Resource": "*"
26 }
27]
28 }

Edit statement

S

Select an exist
ad

+ /



Account Alias

What I did in this step

In this step, I will set up an Alias account, which is like a nickname for our AWS account console's login, because an account alias makes it simpler for our users to log in!

Understanding account aliases

An account alias is simply a nickname for our AWS account.

Setting up my account alias

Creating an account alias took me 30 seconds - it's a simple configuration in the IAM dashboard. Now, my new AWS console sign-in URL uses the alias instead of my account ID.



Create alias for AWS account 875585126274

×

Preferred alias

nextwork-alias-aqib

Must be not more than 63 characters. Valid characters are a-z, 0-9, and - (hyphen).

New sign-in URL

<https://nextwork-alias-aqib.signin.aws.amazon.com/console>

i IAM users will still be able to use the default URL containing the AWS account ID.

[Cancel](#)[Create alias](#)



IAM Users and User Groups

What I did in this step

In this step, I will set up two IAM resources: IAM users and an IAM user group, because IAM users are like logins for people who want access to our AWS account, while users are like folders to manage users that have the same level of access.

Understanding user groups

IAM user groups are like folders that collect IAM users so that you can apply permission settings at the group level.

Attaching policies to user groups

I attached the policy I created to this user group, which means any user created inside this group will automatically get the permissions attached to our NextWorkDevEnvironmentPolicy IAM policy.

Understanding IAM users

An IAM user is an entity you create in AWS to represent a specific person or application that needs to interact with your AWS account. Unlike the root user (which has full, unrestricted access), an IAM user: Has its own unique login credentials (username, password, and sign-in URL). Is granted only the specific permissions they need to do their job (such as accessing development tools but not production resources).



Logging in as an IAM User

Sharing sign-in details

When you successfully create a new user with access to the AWS Management Console, AWS generates their credentials on the final success screen. The two main ways to save and share these details are: Download the .csv file: This file contains the username, password, and the custom console login URL. You can download and securely share this file with your new team member. Copy and paste the credentials: You can manually copy the password and the console sign-in URL directly from the AWS interface and share them via your team's secure password manager, messaging tool, or email.

Observations from the IAM user dashboard

Once I logged in as my IAM user, I noticed that our user had already denied access to panels on the main AWS console dashboard. This was because we only set up permissions for our development EC2 instance, so our intern would not have access to even see anything else



IAM > IAM users > Create user

✔ User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

View user

Step 1

Specify user details

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

Retrieve password

You can view and download the user's password below or email users instructions for signing in to the AWS Management Console. This is the only time you can view and download this password.

Console sign-in details

Email sign-in instructions

Console sign-in URL

https://nextwork-alias-aqib.signin.aws.amazon.com/console

User name

nextwork-dev-aqib

Console password

***** Show

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences



Testing IAM Policies

What I did in this step

In this step, I will log in to our own AWS account as the intern and test access to the production and development instances because we want to make sure that we do not have any ability to do anything that effect our production environment.

Testing policy actions

I attempted to Stop both the production and development EC2 instances while logged in as the restricted IAM user. This action was performed to test whether the custom JSON IAM policy was correctly enforcing access boundaries based on resource tags.

Stopping the production instance

When I tried to stop the production instance, we were met with an error. This was because our production instance is tagged with the 'Production' label, which is outside the scope of our permission policy.



Failed to stop the instance i-08c097f219a296748

Diagnose with Amazon Q

X

You are not authorized to perform this operation. User: arn:aws:iam::875585126274:user/nextwork-dev-aqib is not authorized to perform: ec2:StopInstances on resource: arn:aws:ec2:ap-south-1:875585126274:instance/i-08c097f219a296748 because no identity-based policy allows the ec2:StopInstances action. Encoded authorization failure message: 1molzUb-zgdhkhmsTEYU6LjyA9mUYvSgr2dzA5RxS5Nn4ctyUgvpjKPXuVcx_9To8yXP0ybNmEpmL-JgvMLL-uzs:SIYv__IM4FGKCCk_YLWwv9hJN6cmW2q6R_yJwkv0md_GQsWJlbVetXWLuPDELUWsOu_DW6yB7H6TdGh6N5DuutNnWCB1u9NtHgrT6EXCnwbnbpI5CKWVWpYJ5NJqS6KVdF1IzMARyVw7nDLBP6Cve67HF1A8emUcnloJvLvB8yIETdc4h0e-lkp9PkUvD8aKQVeUFxUuXfUaTl_Q_FhXeOKMqeWpLUatp4F-KAvUtiwS61Zbc_DGlo4SMBFYw_h_9T9scMuZ-KP0GAUICXusJFg2-NehZyK3aQ8Lz56OivwJ2XSI9Xgd6h5JMGcmG3wYnTIHFcfEEJBEF9QtbJhJZMKssrjyLloD6Qop2op2dyCSC8G9JOVMXhx9K1IP1180k4qp_m8kNKF2aVQ7cbawFXHKumq9pAJ1RirFS0IEskKUJ9ZMTO4v6S_J05ZTtygW1oYXS02Wwy4OnlpOyFydqvg8ZA0nxcgNc45AkYQJ5AUygb1nvyDU-rL2rPvRT23BhWkqmrz7pXgl6KJJtE4m2uQ_PxyosSDMb9kgjormXjVeRffhsUzYXGCPeLUQq8lCKprUTTE68FzCVMzvDvsLoly_Pp7V2NKFR1SRMV7JgCP2shgHf7OQGOUSWjhsrV0m4MPUWUJ_S0VBYYW0H7PpUb4o2Lfo7AQx4Ac-uyq2NRF1aFVsmjPD9QDy8au6Vb-SA2SI4bnEjB2rLdwix4DVZb9lqQOCtDovIKtCtKX3UY-6EthzryRekTav_ekW5SuyeCaoA9vxTbICXSYFszLKPUNF85t7loHz02E927v1rCCOK0BNud79cvmmQ

Stopping the development instance

The development instance stopped successfully, and its status changed to Stopping with a green success banner in the AWS console. This happened because the instance had the correct tag (Key: Env, Value: development) which matched the conditional allowance rule inside our IAM policy.

Successfully initiated stopping of i-0d5b984c8ecf7a53d

Instances (1/2) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Choose filter set

Find Instance by attribute or tag (case-sensitive)

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	F
☐	nextwork-pro...	i-08c097f219a296748	Running	t3.micro	3/3 checks passed	ap-south-1b	ec2-13-204-46-30.ap-s...	1
☒	nextwork-dev-...	i-0d5b984c8ecf7a53d	Stopping	t3.micro	-	ap-south-1b	ec2-13-204-77-148.ap-...	1



IAM Policy Simulator

To extend my project, I'm going to test our permission policies in a safer and more controlled way, using a tool called the IAM policy simulator. I'm doing this because having to stop instances and log in to AWS account as other users a bit disruptive.

Understanding the IAM Policy Simulator

The IAM Policy Simulator is a tool that lets us simulate actions and test permissions by defining a specific user/group/role and the action we want to test for. It's useful for saving time when testing permission settings.

How I used the simulator

The Verdict: The action (such as `ec2:StopInstances` or `ec2:StartInstances`) is evaluated as Allowed (indicated by a green success icon). The simulator allows you to safely test these policy actions without affecting live resources. The Triggered Statement: The simulator points directly to your custom IAM policy (specifically Statement 1). This statement allows all EC2 actions on resources tagged with "Env": "development". Condition Verification: Because you simulated the action on a resource containing the key Env and the value development, the condition in your policy was met successfully.



Policy Simulator

Amazon EC2 ▾ 2 Action(s) sele... ▾ **Select All** **Deselect All** **Reset Contexts** **Clear Results** **Run Simulation**

▸ Global Settings ⓘ

Action Settings and Results [2 actions selected. 0 actions not simulated. 1 actions allowed. 1 actions denied.]

	Service	Action	Resource Type	Simulation Resource	Permission
▸	Amazon EC2	StopInstances	instance	*	allowed 1 matching statements.
▸	Amazon EC2	DeleteTags	not required	*	denied 1 matching statements.



nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

